

MÓDULO 5 – Evaluación de Liderazgo Técnico y Gestión

Duración: 30 minutos Evalúa: Liderazgo, priorización, comunicación, capacidad de dirección técnica.

Caso de Liderazgo

Tu equipo tiene 8 desarrolladores. Hay:

- 40% deuda técnica en servicios claves
- 10 incidentes críticos en el último mes
- Presión del negocio para entregar una funcionalidad en 3 semanas
- Falta de estándares y práctica irregular de code review
- Dos desarrolladores junior con brechas técnicas importantes

Preguntas a responder

1. ¿Cuáles serían tus 5 prioridades en las primeras 2 semanas?
2. ¿Cómo organizarías al equipo para mejorar velocidad y calidad?
3. ¿Qué métricas implementarías para evaluar el desempeño del área?
4. ¿Qué prácticas técnicas establecerías como obligatorias?
5. ¿Cómo gestionarías la presión del negocio sin comprometer la calidad?

1. Las 5 prioridades en las primeras 2 semanas

1. Pair Programming: Emparejar a los dos desarrolladores junior con los perfiles más senior, esto para realizar la mentoría estructurada acelera el aprendizaje desde los principios básicos hasta temas avanzados y mitiga el riesgo de errores en producción.
2. Contención de incidentes: estabilizar los servicios clave es vital; no podemos construir sobre un sistema inestable que afecta la operación principal, analizaremos los 10 incidentes del último mes para identificar patrones y atacar la causa raíz más crítica (del más crítico al más débil de los incidentes).
3. Implementación de Code Reviews: Establecer de inmediato un estándar mínimo e **innegociable** de revisión de código para frenar en seco el crecimiento del 40% de deuda técnica que iría de la mano de nuestro 4to punto.
4. Negociación del alcance a 3 semanas: Acordar un MVP (Producto Mínimo Viable) con negocio, dicho objetivo es entregar valor rápido sin comprometer aún más la arquitectura de software y su orden de respetarla hasta donde sea posible.
5. Visibilidad del flujo de trabajo (Kanban/Scrum Mixto): trabajo al máximo para poder mapear todo el trabajo actual (incidentes, deuda, nueva funcionalidad) en un tablero ágil mixto, si no visibilizamos la carga operativa real, no podremos defender los tiempos ante el negocio.

6. Y aunque NO existe un sexto punto, una recompensa a las 3 semanas, NO a las 2 semanas, a mi me gustaria compártir con ellos un viernes (FreeGeek) donde podamos conversar entre nosotros, comer algo (una cerveza o una coca cola, una hamburguesa, y en un ambiente relajado un viernes al terminar o incluso parte del horario laboral para distensionar el ambiente, y hablar de manera más concisa), aunque este seria algo que debemos poner como norma cada mes, eso libera de cualquier tension existente.

2. Organización del equipo para velocidad y calidad

Para evitar el cambio de contexto (que destruye la productividad), implementaría un sistema de "Roles Rotativos":

- Pair Programming (2 personas): asignando a un desarrollador senior y a un junior exclusivamente a soporte, resolución de bugs e incidentes críticos durante una semana (rotando semanalmente), aísla al resto del equipo de las interrupciones.
- La Célula de Entrega (6 personas): Los desarrolladores restantes se enfocan al 100% en la nueva funcionalidad de 3 semanas, trabajarán en parejas (Pair Programming) de manera rotativa para asegurar que el código nace con calidad revisada y para distribuir el conocimiento de los servicios clave, tener dos reuniones una con el equipo junior enmarcada en el conocimiento de la empresa y otra con los senior enmarcada en el respeto para conocer su punto de vista, sus observaciones, ventajas y desventajas y ponernos cada quien a trabajar en el objetivo común.

3. Métricas para evaluar el desempeño del área

No mediremos líneas de código ni horas en la silla, sino el flujo de valor y la estabilidad:

- Tasa de fallos por cambio: esto seria fundamental para saber si nuestras nuevas entregas están causando incidentes, con esto el objetivo es llevar esto a menos del 5%.
- Media estandar de cambio: el tiempo promedio que tardamos en restaurar el servicio cuando ocurre uno de esos incidentes críticos (otra vez del más crítico al menos crítico).
- Lead Time: dentro de la filosofía scrum, tiempo que pasa desde que un desarrollador hace un *commit* hasta que ese código está corriendo exitosamente en producción, saberlo determinar además entendiend que cada commit tiene su complejidad no es lo mismo hacer 10 commit a la semana que 2 commit pero bien hecho, y la complejidad de cada cual,. Nom seria lo mismo como en el caso colocado en el otro factor ([OscarEduardo_OrtizPinzon_versionamiento.pdf](#)) que sea de login que hacer una pantalla de cambio de datos a un programador senior que uno a un junior.
- Proporción de Deuda vs. Valor: al medirse el porcentaje de nuestro tiempo se va en refactorización y bugs versus desarrollo de nuevas características, aunque debo colocar esto como duda tecnica, por que cada desarrollador NO es una maquina que genera código, cada quien responde si se siente bien con su tarea, su deber y para esto se crea el pair programming (para llevar el acompañamiento al desarrollador).

4. Prácticas técnicas obligatorias

Para elevar la calidad sin detener la maquinaria, establecería estas reglas inquebrantables:

- Code Review bloqueante: Ningún PR (Pull Request) pasa a la rama principal (Azure DevOps, GIT, etc.) sin la aprobación de al menos un desarrollador que no sea el autor o del responsable de este grupo de trabajo.
- La Regla del Boy Scout: si cada vez que un desarrollador modifique un servicio clave, debe dejar el código ligeramente más limpio y mejor estructurado de lo que lo encontró. Es la única forma realista de bajar ese 40% de deuda técnica.
- Post-mortems *Blameless* (sin culpas): Por cada nuevo incidente crítico, nos reuniremos para documentar qué falló en el sistema (no quién falló) y crearemos tickets de acción preventiva.
- una recompensa a las 3 semanas, NO a las 2 semanas, a mi me gustaría compartir con ellos un viernes (FreeGeek) donde podamos conversar entre nosotros, comer algo (una cerveza o una coca cola, una hamburguesa, y en un ambiente relajado un viernes al terminar o incluso parte del horario laboral para distensionar el ambiente, y hablar de manera más concisa), aunque este sería algo que debemos poner como norma cada mes, eso libera de cualquier tensión existente.

5. Gestión de la presión del negocio

La clave aquí es la transparencia radical basada en datos.

- Dentro del negocio a menudo no entiende qué es la "deuda técnica", existe que hay explicárselos en su idioma: los 10 incidentes críticos representan horas de inactividad, mala experiencia de usuario y dinero perdido, la deuda técnica es un préstamo con intereses altísimos que ralentiza la entrega de sus nuevas ideas (que no nos pase como en Bancolombia).
- En lugar de decir "no podemos hacerlo en 3 semanas, sería mejor si vamos a realizar en 3 semanas", respuesta debe ser: *"En 3 semanas podemos entregar la funcionalidad A y B con calidad, Si forzamos la inclusión de C, el riesgo de sumar incidentes a los 10 actuales es crítico."*
- Y lo resalto, sea como sea todo sacrificio debe tener su recompensa FreeGeek sería la mejor alternativa, todo NO debe ser trabajar y no poder disfrutar lo que se hace.